

Sistema de organización de torneos de videojuegos: Arquitectura, Escalabilidad y Gestión de Pagos

Daniel Alejandro Melendez Delgado Samuel Abraham Rodriguez Garay Diana Michell Velasquez Melendez
Gestión de Torneos y Participantes Gestión de Pagos e Integración Stripe Brackets, Resultados y Seguridad

Resumen—Este documento detalla la arquitectura, las decisiones técnicas y el flujo de trabajo colaborativo implementados en el “Sistema de Organización de Torneos de Videojuegos” (Proyecto 11). Se presenta una justificación exhaustiva de la arquitectura de software basada en N-Capas sobre el ecosistema Spring Boot, la aplicación del patrón de diseño Strategy para resolver el desafío de escalabilidad de formatos de torneo, la integración segura de la pasarela de pagos Stripe y el uso de una base de datos PostgreSQL alojada en la nube (Render). Asimismo, se documenta la metodología de control de versiones con Git, en la cual la estructura de ramas se definió de acuerdo con la asignación de responsabilidades de cada integrante del equipo, garantizando un entorno de integración limpio en la rama `develop`.

Index Terms—Arquitectura N-Capas, Spring Boot, Patrón Strategy, Stripe, GitFlow, eSports.

I. INTRODUCCIÓN

El auge de los deportes electrónicos ha generado una demanda creciente de plataformas robustas capaces de gestionar competiciones a gran escala. El presente proyecto aborda esta necesidad mediante el desarrollo de un backend integral diseñado para la creación de torneos, inscripción de jugadores, generación dinámica de emparejamientos (brackets), registro de resultados, gestión de premios y procesamiento de pagos.

El principal desafío técnico del proyecto radicó en la necesidad de soportar múltiples formatos de competición (eliminatória simple, doble, liga, suizo, round robin) y de escalar el sistema sin duplicar la lógica de negocio ni comprometer el rendimiento. El desarrollo se organizó en tres módulos funcionales principales, asignados a los integrantes del equipo, y se coordinó mediante un flujo de trabajo estructurado en Git.

II. ARQUITECTURA DE SOFTWARE

El sistema fue diseñado utilizando una **Arquitectura N-Capas (N-Tier Architecture)** basada en el framework Spring Boot. Esta decisión arquitectónica se fundamenta en el principio de separación de responsabilidades, lo que facilita el mantenimiento, la prueba unitaria y la escalabilidad del sistema [1].

II-A. Capa de Presentación (Controllers)

Esta capa es el punto de entrada de las peticiones HTTP. Los controladores (por ejemplo, `TorneoController`, `BracketController`, `PagoController`, `DisputaController`) están anotados con `@RestController` y se encargan exclusivamente de interceptar las peticiones REST, validar

las entradas de datos mediante `jakarta.validation (@Valid)`, delegar la lógica a la capa de servicios y formatear las respuestas utilizando un envoltorio estandarizado (`GeneralResponse`).

II-B. Capa de Negocio (Services)

Representa el núcleo de la aplicación. Las clases anotadas con `@Service` (por ejemplo, `TorneoService`, `BracketService`, `ResultadoService`, `PagoService`) contienen las reglas de negocio. Esta capa asegura que las operaciones sean atómicas y consistentes mediante el uso de la anotación `@Transactional`, la cual gestiona los límites de las transacciones de base de datos [2].

II-C. Capa de Acceso a Datos (Repositories)

Implementada mediante Spring Data JPA. Las interfaces que extienden `JpaRepository` (por ejemplo, `TorneoRepository`, `UsuarioRepository`, `PagoRepository`) proporcionan abstracciones de alto nivel para realizar operaciones CRUD y consultas personalizadas, aislando a la capa de negocio de la sintaxis SQL subyacente.

II-D. Capa de Transferencia de Datos (DTOs y Mappers)

Para evitar la exposición directa de las entidades de la base de datos y prevenir problemas de carga perezosa durante la serialización JSON, se implementó un riguroso patrón DTO (*Data Transfer Object*). Se utilizaron mapeadores dedicados (`TorneoMapper`, `BracketMapper`, `ResultadoMapper`, `DisputaMapper`, `UsuarioMapper`) para transformar los datos entre las capas de presentación y persistencia de forma segura.

II-E. Infraestructura y Persistencia en la Nube

La persistencia de datos se implementó sobre **PostgreSQL**, un sistema de gestión de bases de datos relacional robusto y de código abierto. La instancia de base de datos se desplegó en **Render**, una plataforma de servicios en la nube, lo que permitió que todos los integrantes del equipo trabajaran contra una misma fuente de datos centralizada y persistente durante el desarrollo. La configuración de la conexión y las claves de servicios externos se externalizaron mediante variables de entorno, evitando su exposición directa en el código fuente.

III. MÓDULOS FUNCIONALES Y ASIGNACIÓN DE RESPONSABILIDADES

El proyecto se dividió en tres grandes módulos funcionales, cada uno asignado a un integrante del equipo. Esta división guió tanto el desarrollo como la estrategia de ramificación en Git.

III-A. Daniel — Gestión de Torneos y Participantes

Este módulo constituye la base de la plataforma. Comprende la API para la creación de torneos (definiendo juego, formato y número de participantes) y su gestión completa (actualizar, eliminar y consultar detalles), implementada en `TorneoController` y `TorneoService`. Incluye la lógica de configuración para los distintos formatos de torneo, la validación de rangos ELO para la inscripción y la lógica del **límite de participantes** por torneo. En el módulo de participantes se implementó la API para la inscripción y gestión de jugadores (añadir y remover), junto con un sistema de notificaciones simulado que informa a los participantes sobre sus próximos partidos.

III-B. Michell — Brackets, Resultados, Escalabilidad y Seguridad

Este módulo aborda el corazón competitivo del sistema. Incluye la API para la **generación automática de brackets** (aleatoria o por ranking), la lógica de programación de fechas y horarios por ronda (`ProgramacionService`) y la exportación de brackets a imagen/PDF (`ExportacionBracketService`). El módulo de resultados gestiona el registro de ganadores y marcadores con validación, así como el **sistema de disputas** para que el administrador revise y resuelva los conflictos (`DisputaService`). Adicionalmente, este integrante fue responsable de la implementación de la **capa de seguridad transversal** basada en JWT (descrita en la Sección V-B). La solución al desafío de escalabilidad de formatos, mediante el patrón Strategy, corresponde también a este módulo y se detalla en la Sección IV.

III-C. Samuel — Perfiles, Premios, Reportes y Pagos

Este módulo gestiona la identidad de los usuarios y la economía de la plataforma. Comprende la API para la gestión de perfiles con roles diferenciados (Jugador, Organizador, Administrador), la API para el historial de torneos y estadísticas de jugadores (`TorneoEstadisticasService`) y el **ranking global** de jugadores. En el módulo de premios y puntos se implementó la gestión de créditos virtuales o físicos (`PremioService`, `PuntoService`) y la lógica de asignación de premios al finalizar los torneos. El módulo de reportes genera informes de los torneos más populares por juego y estadísticas de jugadores (`ReporteService`). Finalmente, este integrante fue responsable de la **integración de la pasarela de pagos Stripe** (detallada en la Sección VI).

IV. SOLUCIÓN AL DESAFÍO DE ESCALABILIDAD: PATRÓN STRATEGY

El requerimiento de soportar múltiples formatos de torneo y de escalar potencialmente a decenas de formatos presentaba un riesgo de acoplamiento severo si se implementaba mediante estructuras condicionales masivas (`if-else` o `switch`).

Para resolver este desafío se implementó el **Patrón Strategy**. Se definió una interfaz común `FormatoStrategy` con tres operaciones fundamentales:

```
public interface FormatoStrategy {  
    void validarConfiguracion(Torneo t);  
    Bracket generarBracketInicial(Torneo t,  
        List<Usuario> part);  
    void avanzarRonda(Bracket b);  
}
```

Cada formato posee su propia implementación concreta (`EliminatoriaSimpleStrategy`, `EliminatoriaDobleStrategy`, `LigaStrategy`, `RoundRobinStrategy`, `SuizoStrategy`), encapsulando su lógica algorítmica específica de generación de emparejamientos y avance de rondas. La clase de configuración `EstrategiaFormatoConfig` registra estas estrategias en un `Map<FormatoTorneo, FormatoStrategy>`, lo que permite al `TorneoService` y al `BracketService` seleccionar la estrategia adecuada en tiempo de ejecución con complejidad $O(1)$.

Esta decisión técnica cumple con el **Principio Abierto/Cerrado (Open/Closed Principle)** de SOLID [3].

V. DECISIONES TÉCNICAS TRANSVERSALES

V-A. Roles y Autorización

Se definieron tres roles en el enumerador `RolUsuario`: `ADMINISTRADOR`, `ORGANIZADOR` y `JUGADOR`. El administrador crea torneos globales, resuelve disputas y gestiona premios; el organizador configura formatos, fechas e invita participantes; y el jugador se inscribe, reporta resultados y consulta brackets.

V-B. Seguridad Transversal con JWT

La seguridad se implementó con **Spring Security** y **JSON Web Tokens** bajo una arquitectura *stateless*. El `AuthService` verifica las credenciales y emite un JWT firmado mediante HMAC-SHA256 (`JwtUtil`). El filtro personalizado `JwtAuthFilter` intercepta cada petición, valida la firma y expiración del token, y reconstruye el contexto de seguridad inyectando los roles (con prefijo `ROLE_`). Las contraseñas se almacenan cifradas con `BCryptPasswordEncoder`.

V-C. Manejo Global de Excepciones

Se centralizó la captura de errores mediante la clase `GlobalExceptionHandler` anotada con `@RestControllerAdvice`. Esto garantiza que cualquier excepción de negocio o de seguridad sea traducida en una respuesta HTTP semánticamente correcta (400, 401, 403, 404, 409, 500).

VI. INTEGRACIÓN DE PASARELA DE PAGOS (STRIPE)

Para gestionar la compra de créditos virtuales y el pago de inscripciones a torneos premium, se integró la pasarela de pagos **Stripe**.

VI-A. Flujo de Intento de Pago

El `PagoService` no procesa datos de tarjetas directamente. En su lugar, se comunica con la API de Stripe mediante el `StripeService` para crear un `PaymentIntent`. El backend almacena un registro local del pago en estado `PENDIENTE` y devuelve al cliente un `client_secret` para completar la transacción de forma segura directamente con los servidores de Stripe.

VI-B. Webhooks e Idempotencia

Para confirmar que un pago se ha realizado con éxito, el sistema expone un endpoint seguro (`POST /api/pagos/webhook`) en el `PagoController`:

1. **Verificación Criptográfica:** El webhook valida la firma digital de la cabecera `Stripe-Signature`.
2. **Idempotencia:** Antes de procesar un evento, el sistema verifica en la base de datos si el `stripeEventId` ya ha sido procesado mediante la entidad `WebhookStripe`.
3. **Acceso Condicional:** Tras validar el pago, el evento dispara la actualización del estado a `COMPLETADO`.

VII. FLUJO DE TRABAJO Y CONTROL DE VERSIONES

El desarrollo se coordinó utilizando **Git** con un repositorio remoto alojado en GitHub (`Grupo6_pnc_backend`), bajo un modelo de ramificación inspirado en *GitFlow* [4]. La estructura de ramas se definió de acuerdo con la parte del proyecto asignada a cada integrante.

VIII. PRUEBAS Y VALIDACIÓN DE LA API

La validación funcional de los endpoints REST se realizó utilizando **Postman**. Esta herramienta permitió construir una colección de peticiones para probar de forma sistemática cada uno de los módulos del sistema.

IX. CONCLUSIÓN

La arquitectura N-Capas, combinada con el patrón Strategy, dotó al Sistema de Organización de Torneos de una base altamente modular y escalable. La integración de Stripe y el despliegue de PostgreSQL en Render aseguran un manejo transaccional confiable, coordinado eficientemente mediante una metodología disciplinada en Git.

REFERENCIAS

- (1) “N-Tier Architecture Style”, Microsoft Azure Architecture Center. Disponible: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier>
- (2) “Transaction Management”, Spring Framework Reference Documentation. Disponible: <https://docs.spring.io/spring-framework/reference/data-access/transaction.html>
- (3) E. Gamma, R. Helm, R. Johnson, y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.
- (4) V. Driessen, “A successful Git branching model”, nvie.com, 2010. Disponible: <https://nvie.com/posts/a-successful-git-branching-model/>